02-06接口自动化Agent开发之框架设计

一、接口自动化基础和框架

1、requests 请求库

掌握 Python `requests` 库的常用方法，完成基本的接口调用。

1.1 `requests` 简介与安装

- pip 安装方式
- 简要对比 urllib、httpx

1.2 常用方法与示例

- `GET`、`POST`、`PUT`、`DELETE` 请求
- 传参方式（params、data、json）
- 请求头（headers）、cookies 的设置
- 文件上传与下载
- 超时、重试、异常处理

1.3 实战演练

- 以在线 mock 接口（如 httpbin 或 apifox）为例，构建请求封装函数

封装一个基础版 `RequestHandler` 类，支持自动打印日志、异常处理。

2、pytest 框架

掌握如何使用 pytest 编写可维护、可执行、可复用的测试用例。

2.1 pytest 简介与优势

- 与 unittest 的对比
- 插件生态丰富（如 pytest-html、pytest-xdist）

2.2 编写基本用例

- 使用 `assert` 编写断言
- 用例组织结构（文件夹、模块、函数）

2.3 参数化测试

- `@pytest.mark.parametrize`
- 数据驱动思想结合 yaml/csv/json

2.4 setup/teardown

- 函数级、模块级、类级、session级

2.5 用例分组与执行控制

- `mark` 标签与选择执行
- 跳过、失败重试（`skip`，`xfail`，`rerunfailures`）

3、allure报告

3.1 allure 安装与环境配置

- 安装命令

3.2 allure 与 pytest 结合

- 添加依赖：`pytest-allure-adaptor` 或 `allure-pytest`
- 生成 XML 报告与 HTML 报告

3.3 allure 常用注解

- `@allure.feature`
- `@allure.story`
- `@allure.step`
- `@allure.title`，`@allure.description`

4、日志模块

4.1 日志系统 logging

- 日志级别（DEBUG/INFO/WARNING/ERROR/CRITICAL）
- 输出格式、文件保存
- 日志按时间滚动保存（RotatingFileHandler）

4.2 日志封装

- 创建 `Logger` 工具类
- 支持多模块复用与自定义格式

二、接口自动化框架封装

1、BaseTestCase封装

```
# @Author  : 木森
# @weixin: python771
# base/base_case.py
import re
from numbers import Number

import requests
import jmespath
import allure
import pymysql
import json
```

```python
from common.logger import Logger  # 导入日志模块


class BaseTestCase:
    def __init__(self, case_data):
        self.case_data = case_data
        self.session = requests.Session()
        self.variables = {}  # 存储提取变量
        self.logger = Logger(__name__)  # 初始化日志对象

    def _replace_vars(self, data):
        """替换变量"""
        pattern = r'{{(.+?)}}'
        old_data = data
        """解析变量"""
        if isinstance(data, str):
            while re.search(pattern, data):
                res2 = re.search(pattern, data)
                item = res2.group()
                attr = res2.group(1)
                value = self.variables.get(attr)
                if value is None:
                    raise ValueError('变量引用错误:\n{}中的变量{},在当前运行环境中未找
到'.format(data, attr))
                if item == data: return value
                data = data.replace(item, str(value))
                allure.attach(f"变量: {attr} = {value}", name=f"替换变量{attr}",
                              attachment_type=allure.attachment_type.TEXT)
                self.logger.info(f"替换变量: {attr} = {value}")
            return data
        elif isinstance(data, list) or isinstance(data, dict) or
isinstance(data, tuple) or isinstance(data, set):
            data = str(data)
            while re.search(pattern, data):
                res2 = re.search(pattern, data)
                item = res2.group()
                attr = res2.group(1)
                value = self.variables.get(attr)
                if value is None:
                    raise ValueError('变量引用错误: \n{}\n中的变量{},在当前运行环境的测
试数据中未找到'.format(
                        json.dumps(old_data, ensure_ascii=False, indent=2),
attr)
                    )
                if isinstance(value, Number):
                    s = data.find(item)
                    dd = data[s - 1:s + len(item) + 1]
                    data = data.replace(dd, str(value))
                elif isinstance(value, str) and "'" in value:
                    data = data.replace(item, value.replace("'", '"'))
                else:
                    data = data.replace(item, str(value))
                allure.attach(f"变量: {attr} = {value}", name=f"替换变量{attr}",
                              attachment_type=allure.attachment_type.TEXT)
                self.logger.info(f"替换变量: {attr} = {value}")
            return eval(data)
        else:
            return data
```

```
    def _send_request(self, request_data):
        """请求发送的方法"""
        method = request_data["method"].upper()
        url = request_data["base_url"].rstrip("/") + request_data["url"]
        headers = self._replace_vars(request_data.get("headers", {}))
        self.logger.info(f"替换之后的请求头：{headers}")
        params = self._replace_vars(request_data.get("params", {}))
        body = self._replace_vars(request_data.get("body", {}))
        # 请求内容记录
        request_info = {
            "method": method,
            "url": url,
            "headers": headers,
            "params": params,
            "body": body
        }
        allure.attach(json.dumps(request_info, indent=2, ensure_ascii=False),
name="请求信息",
                      attachment_type=allure.attachment_type.JSON)
        self.logger.info(f"发送请求: {method} {url}")  # 添加日志记录

        # 判断 Content-Type 决定如何发送 body
        content_type = headers.get("Content-Type", "").lower()
        request_kwargs = {
            "method": method,
            "url": url,
            "headers": headers,
            "params": params,
            "timeout": 10
        }

        if method in ["POST", "PUT", "PATCH"]:
            if "application/json" in content_type:
                request_kwargs["json"] = body
            elif "application/x-www-form-urlencoded" in content_type:
                request_kwargs["data"] = body
            elif "multipart/form-data" in content_type:
                request_kwargs["files"] = body
            else:
                # 未指定 Content-Type，默认用 json
                request_kwargs["json"] = body

        try:
            response = self.session.request(**request_kwargs)
            self.logger.info(f"收到响应: 状态码 {response.status_code}")  # 添加日志
记录
        except Exception as e:
            allure.attach(str(e), name="请求异常",
attachment_type=allure.attachment_type.TEXT)
            self.logger.error(f"请求异常: {str(e)}")  # 添加日志记录
            raise

        # 响应内容记录
        try:
            resp_json = response.json()
            allure.attach(json.dumps(resp_json, indent=2, ensure_ascii=False),
name="响应内容",
```

```
                            attachment_type=allure.attachment_type.JSON)
        except Exception:
            allure.attach(response.text, name="响应内容（非 JSON）",
attachment_type=allure.attachment_type.TEXT)

        return response

    def _extract_variables(self, response, extract_rules):
        for var_name, jmes_expr in extract_rules:
            value = jmespath.search(jmes_expr, response.json())
            self.variables[var_name] = value
            allure.attach(f"提取变量: {var_name} = {value}", name=f"提取变量
{var_name}",
                            attachment_type=allure.attachment_type.TEXT)
            self.logger.info(f"提取变量: {var_name} = {value}")  # 添加日志记录

    def _assert_response(self, response, assertions):
        for assertion in assertions:
            field = assertion["field"]
            expected = assertion.get("expected")

            if assertion["type"] == "http_code":
                # 记录断言信息到allure
                allure.attach(f"断言HTTP状态码: {field}\n预期值: {expected}\n实际值:
{response.status_code}",
                              name="断言信息", )
                assert response.status_code == expected, f"[{field}] expect:
{expected}, got: {response.status_code}"
                self.logger.info(
                    f"断言HTTP状态码成功: [{field}] expect: {expected}, got:
{response.status_code}")  # 添加日志记录
                continue
            elif assertion["type"] == "equal":
                actual = jmespath.search(field, response.json())
                allure.attach(f"断言字段: {field}\n预期值: {expected}\n实际值:
{actual}",
                              name="断言信息", )
                assert actual == expected, f"[{field}] expect: {expected}, got:
{actual}"
                self.logger.info(f"断言成功: [{field}] expect: {expected}, got:
{actual}")  # 添加日志记录
            elif assertion["type"] == "not_null":
                actual = jmespath.search(field, response.json())
                allure.attach(f"断言断言字段: {field}\n预期值: {expected}\n实际值:
{actual}",
                              name="断言信息", )
                assert actual is not None, f"[{field}] expected not null, got:
{actual}"
                self.logger.info(f"断言成功: [{field}] 不为空")  # 添加日志记录

    def _assert_database(self, db_assertions):
        # 此处为简化演示，建议将 DB 配置移入 config 文件中
        conn = pymysql.connect(
            host="localhost",
            user="root",
            password="mysql",
            database="test",
            cursorclass=pymysql.cursors.DictCursor
```

```
        )
        with conn.cursor() as cursor:
            for db_check in db_assertions:
                query = self._replace_vars(db_check["query"])
                cursor.execute(query)
                result = cursor.fetchone()
                for k, v in db_check["expected"].items():
                    actual = result.get(k)
                    assert actual == v, f"[DB] field {k}: expected {v}, got
{actual}"
                    self.logger.info(f"数据库断言成功: 字段 {k} 预期值 {v}, 实际值
{actual}")  # 添加日志记录
        conn.close()

    def run(self):
        with allure.step("前置步骤"):
            for step in self.case_data.get("preconditions", []):
                name = step["name"]
                with allure.step(name):
                    response = self._send_request(step["request"])
                    if "extract" in step:
                        self._extract_variables(response, step["extract"])

        with allure.step("执行主接口请求"):
            main_request = self.case_data["request"]
            response = self._send_request(main_request)

        with allure.step("响应断言"):
            assertions = self.case_data.get("assertions", {}).get("response",
[])
            assertions = self._replace_vars(assertions)
            self._assert_response(response, assertions)
        # 判断是否有数据库断言
        if self.case_data.get("assertions", {}).get("database", []):
            with allure.step("数据库断言"):
                db_assertions = self.case_data.get("assertions",
{}).get("database", [])
                db_assertions = self._replace_vars(db_assertions)
                self._assert_database(db_assertions)
```

2、日志处理封装

```
import logging
import os


class Logger:
    _instance = None  # 单例模式的核心变量

    def __new__(cls, *args, **kwargs):
        # 如果实例尚未创建，则创建一个新的实例
        if cls._instance is None:
            cls._instance = super(Logger, cls).__new__(cls)
            cls._instance.__initialized = False  # 防止重复初始化
        return cls._instance
```

```
    def __init__(self, name, level=logging.DEBUG):
        # 防止重复初始化
        if self.__initialized:
            return
        self.logger = logging.getLogger(name)
        self.logger.setLevel(level)

        # 创建控制台处理器
        console_handler = logging.StreamHandler()
        console_handler.setLevel(level)

        # 创建格式化器
        formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s -
%(message)s')
        console_handler.setFormatter(formatter)
        # 添加处理器到日志器
        self.logger.addHandler(console_handler)

        self.__initialized = True  # 标记已初始化

    def info(self, message):
        self.logger.info(message)

    def debug(self, message):
        self.logger.debug(message)

    def warning(self, message):
        self.logger.warning(message)

    def error(self, message):
        self.logger.error(message)

    def critical(self, message):
        self.logger.critical(message)


# 示例用法
if __name__ == "__main__":
    logger1 = Logger(__name__, log_file='app.log')
    logger2 = Logger(__name__, log_file='app.log')  # 再次创建时会复用之前的实例
    logger1.info("这是一个信息日志")
    logger2.debug("这是一个调试日志")
```

3、通用执行的测试用例封装

```
import pytest
import allure
from common.base_cases import BaseTestCase

from data.test_user_data import user_data

@pytest.mark.parametrize("case_data", user_data)
def test_case(case_data):
    with allure.step(f"执行用例： {case_data['name']}"):
```

```
        allure.dynamic.epic(case_data.get("project", "项目名称"))
        allure.dynamic.feature(case_data.get("module", "默认模块"))
        allure.dynamic.story(case_data.get("interface", "接口名称"))
        allure.dynamic.title(case_data.get("name", "测试用例"))
        case = BaseTestCase(case_data)
        case.run()
```